Operating Systems Lab

Program 1: CPU Scheduling | Program 2: System Calls

Question

1. CPU Scheduling Algorithms

i) A washing machine which requires the process to be executed sequentially. Consider the processes P1, P2, P3, P4 whose arrival times are 1, 5, 9, 10 and burst times are 4, 3, 5, 2 respectively. Implement an appropriate algorithm. Find the CPU idle time, so that the water can be supplied during that period of time.

ii) Implement shortest job first for the following data: Consider the following set of processes, CPU burst time, Arrival time. Calculate the average waiting time, average response time and average turnaround time.

Process	Burst Time	Arrival Time
P1	3	0
P2	6	2
P3	4	4
P4	5	6
P5	2	8

iii) Implement Round Robin for the following data. Consider the following set of processes and length of the CPU burst time given in milliseconds.

Process	Burst Time
P1	10
P2	1
P3	2
P4	1
P5	5

The processes are assumed to have arrived in the order P1, P2, P3, P4, P5 all at time 0 and time quantum in RR = 1. Calculate the average waiting time, response time and turnaround time.

Display the gantt charts also.

2. Process Management using System Calls: Fork, Exec, Getpid, Exit, Wait, Close

Program 1: CPU Scheduling Algorithms

FCFS (washing machine with idle time), Non-Preemptive SJF and Round Robin, selected from a menu.

```
from collections import deque


def display_results(results):
    print("\nPROCESS DETAILS")
    print("-" * 95)
    print(
        f"{'Process':<10}{'Arrival':<10}{'Burst':<10}"
        f"{'Start':<10}{'Completion':<13}{'Waiting':<10}"
        f"{'Turnaround':<13}{'Response':<10}"
    )
    print("-" * 95)

    for r in results:
        print(
            f"{r['pid']:<10}{r['arrival']:<10}"
            f"{r['burst']:<10}{r['start']:<10}"
            f"{r['completion']:<13}{r['waiting']:<10}"
            f"{r['turnaround']:<13}{r['response']:<10}"
        )

    print("-" * 95)

    if results:
        n = len(results)
        print(f"Average Waiting Time    : {sum(r['waiting'] for r in results) / n:.2f}")
        print(f"Average Turnaround Time : {sum(r['turnaround'] for r in results) / n:.2f}")
        print(f"Average Response Time   : {sum(r['response'] for r in results) / n:.2f}")


def display_gantt_chart(chart):
    print("\nGANTT CHART")

    for start, end, process in chart:
        print(f"| {process} ", end="")
    print("|")

    print("Timeline:")
    print(chart[0][0], end=" ")

    for start, end, process in chart:
        print(end, end=" ")

    print("\n")


def fcfs_washing_machine():
    print("\nFCFS CPU SCHEDULING - WASHING MACHINE")

    processes = [
        {"pid": "P1", "arrival": 1, "burst": 4},
        {"pid": "P2", "arrival": 5, "burst": 3},
        {"pid": "P3", "arrival": 9, "burst": 5},
        {"pid": "P4", "arrival": 10, "burst": 2}
    ]

    processes.sort(key=lambda p: p["arrival"])

    current_time = 0
    total_idle_time = 0
    chart = []
    results = []
```

```
    for p in processes:
        arrival = p["arrival"]
        burst = p["burst"]

        if current_time < arrival:
            chart.append((current_time, arrival, "IDLE"))
            total_idle_time += arrival - current_time
            current_time = arrival

        start = current_time
        completion = start + burst

        results.append({
            "pid": p["pid"],
            "arrival": arrival,
            "burst": burst,
            "start": start,
            "completion": completion,
            "waiting": start - arrival,
            "turnaround": completion - arrival,
            "response": start - arrival
        })

        chart.append((start, completion, p["pid"]))
        current_time = completion

    display_results(results)
    display_gantt_chart(chart)

    print("Total CPU Idle Time:", total_idle_time, "time units")


def sjf_scheduling():
    print("\nNON-PREEMPTIVE SJF CPU SCHEDULING")

    processes = [
        {"pid": "P1", "arrival": 0, "burst": 3},
        {"pid": "P2", "arrival": 2, "burst": 6},
        {"pid": "P3", "arrival": 4, "burst": 4},
        {"pid": "P4", "arrival": 6, "burst": 5},
        {"pid": "P5", "arrival": 8, "burst": 2}
    ]

    current_time = 0
    completed = []
    chart = []
    results = []

    while len(completed) < len(processes):
        available = [
            p for p in processes
            if p["pid"] not in completed
            and p["arrival"] <= current_time
        ]

        if not available:
            next_arrival = min(
                p["arrival"] for p in processes
                if p["pid"] not in completed
            )
            chart.append((current_time, next_arrival, "IDLE"))
            current_time = next_arrival
            continue

        # Select the shortest burst among arrived processes
        p = min(available, key=lambda x: (x["burst"], x["arrival"]))
```

```
        start = current_time
        completion = start + p["burst"]

        results.append({
            "pid": p["pid"],
            "arrival": p["arrival"],
            "burst": p["burst"],
            "start": start,
            "completion": completion,
            "waiting": start - p["arrival"],
            "turnaround": completion - p["arrival"],
            "response": start - p["arrival"]
        })

        chart.append((start, completion, p["pid"]))
        current_time = completion
        completed.append(p["pid"])

    display_results(results)
    display_gantt_chart(chart)


def round_robin():
    print("\nROUND ROBIN CPU SCHEDULING")

    processes = [
        {"pid": "P1", "arrival": 0, "burst": 10},
        {"pid": "P2", "arrival": 0, "burst": 1},
        {"pid": "P3", "arrival": 0, "burst": 2},
        {"pid": "P4", "arrival": 0, "burst": 1},
        {"pid": "P5", "arrival": 0, "burst": 5}
    ]

    quantum = 1
    remaining = {p["pid"]: p["burst"] for p in processes}
    first_start = {}
    completion_times = {}
    current_time = 0
    queue = deque(p["pid"] for p in processes)
    chart = []

    while queue:
        pid = queue.popleft()
        p = next(p for p in processes if p["pid"] == pid)

        if pid not in first_start:
            first_start[pid] = current_time

        run_time = min(quantum, remaining[pid])
        start = current_time
        current_time += run_time
        remaining[pid] -= run_time

        chart.append((start, current_time, pid))

        if remaining[pid] > 0:
            queue.append(pid)
        else:
            completion_times[pid] = current_time

    results = []

    for p in processes:
        pid = p["pid"]
        arrival = p["arrival"]
        burst = p["burst"]
```

```
        completion = completion_times[pid]
        start = first_start[pid]

        results.append({
            "pid": pid,
            "arrival": arrival,
            "burst": burst,
            "start": start,
            "completion": completion,
            "waiting": completion - arrival - burst,
            "turnaround": completion - arrival,
            "response": start - arrival
        })

    display_results(results)
    display_gantt_chart(chart)
    print("Time Quantum:", quantum, "ms")


def main():
    while True:
        print("\nCPU SCHEDULING MENU")
        print("1. FCFS - Washing Machine")
        print("2. Non-Preemptive SJF")
        print("3. Round Robin")
        print("4. Exit")

        choice = input("Enter your choice (1-4): ")

        if choice == "1":
            fcfs_washing_machine()
        elif choice == "2":
            sjf_scheduling()
        elif choice == "3":
            round_robin()
        elif choice == "4":
            print("Program terminated.")
            break
        else:
            print("Invalid choice. Please enter 1, 2, 3, or 4.")


if __name__ == "__main__":
    main()
```

Output of Program 1

Output 1: Option 1, FCFS (Washing Machine)

```
CPU SCHEDULING MENU
1. FCFS - Washing Machine
2. Non-Preemptive SJF
3. Round Robin
4. Exit
Enter your choice (1-4): 1

FCFS CPU SCHEDULING - WASHING MACHINE

PROCESS DETAILS
-----------------------------------------------------------------------------------------------
Process   Arrival   Burst     Start     Completion   Waiting   Turnaround   Response
-----------------------------------------------------------------------------------------------
P1        1         4         1         5            0         4            0
P2        5         3         5         8            0         3            0
P3        9         5         9         14           0         5            0
P4        10        2         14        16           4         6            4
-----------------------------------------------------------------------------------------------
Average Waiting Time    : 1.00
Average Turnaround Time : 4.50
Average Response Time   : 1.00

GANTT CHART
| IDLE | P1 | P2 | IDLE | P3 | P4 |
Timeline:
0 1 5 8 9 14 16

Total CPU Idle Time: 2 time units
```

Output 2: Option 2, Non-Preemptive SJF

```
CPU SCHEDULING MENU
1. FCFS - Washing Machine
2. Non-Preemptive SJF
3. Round Robin
4. Exit
Enter your choice (1-4): 2

NON-PREEMPTIVE SJF CPU SCHEDULING

PROCESS DETAILS
-----------------------------------------------------------------------------------------------
Process   Arrival   Burst     Start     Completion   Waiting   Turnaround   Response
-----------------------------------------------------------------------------------------------
P1        0         3         0         3            0         3            0
P2        2         6         3         9            1         7            1
P5        8         2         9         11           1         3            1
P3        4         4         11        15           7         11           7
P4        6         5         15        20           9         14           9
-----------------------------------------------------------------------------------------------
Average Waiting Time    : 3.60
Average Turnaround Time : 7.60
Average Response Time   : 3.60

GANTT CHART
| P1 | P2 | P5 | P3 | P4 |
Timeline:
0 3 9 11 15 20
```

Output 3: Option 3, Round Robin (Time Quantum = 1 ms)

```
CPU SCHEDULING MENU
1. FCFS - Washing Machine
2. Non-Preemptive SJF
3. Round Robin
4. Exit
Enter your choice (1-4): 3

ROUND ROBIN CPU SCHEDULING

PROCESS DETAILS
-----------------------------------------------------------------------------------------------
Process   Arrival   Burst     Start     Completion   Waiting   Turnaround   Response
-----------------------------------------------------------------------------------------------
P1        0         10        0         19           9         19           0
P2        0         1         1         2            1         2            1
P3        0         2         2         7            5         7            2
P4        0         1         3         4            3         4            3
P5        0         5         4         14           9         14           4
-----------------------------------------------------------------------------------------------
Average Waiting Time    : 5.40
Average Turnaround Time : 9.20
Average Response Time   : 2.00

GANTT CHART
| P1 | P2 | P3 | P4 | P5 | P1 | P3 | P5 | P1 | P5 | P1 | P5 | P1 | P5 | P1 | P1 | P1 | P1 | P1 |
Timeline:
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19

Time Quantum: 1 ms
```

Output 4: Invalid choice and Exit

```
CPU SCHEDULING MENU
1. FCFS - Washing Machine
2. Non-Preemptive SJF
3. Round Robin
4. Exit
Enter your choice (1-4): 7
Invalid choice. Please enter 1, 2, 3, or 4.

CPU SCHEDULING MENU
1. FCFS - Washing Machine
2. Non-Preemptive SJF
3. Round Robin
4. Exit
Enter your choice (1-4): 4
Program terminated.
```

Program 2: Process Management using System Calls

Filename: `02_system_calls.py`

```
import os
import sys
import subprocess
import multiprocessing


def child_process():
    print("\nChild Process")
    print("Child Process ID:", os.getpid())
    print("Parent Process ID:", os.getppid())
    print("Executing directory listing command...")

    # exec-like demonstration: launch another program
    result = subprocess.run(
        ["cmd", "/c", "dir"],
        check=False
    )

    # Exit status
    sys.exit(result.returncode)


if __name__ == "__main__":
    print("PROCESS MANAGEMENT USING SYSTEM CALLS")
    print("----------------------------------------")

    # getpid()
    print("Parent Process ID:", os.getpid())

    # close()
    fd = os.open(
        "sample.txt",
        os.O_CREAT | os.O_WRONLY | os.O_TRUNC,
        0o644
    )

    os.write(fd, b"Operating Systems Lab\n")
    os.close(fd)

    print("File created and file descriptor closed.")

    # Windows-compatible child process creation
    process = multiprocessing.Process(target=child_process)
    process.start()

    print("\nParent Process")
    print("Created Child Process ID:", process.pid)

    # wait()
    process.join()

    print("\nChild process completed.")
    print("Child Exit Status:", process.exitcode)
    print("Parent process terminates normally.")
```

System calls used in the program

System call	Where it appears in the program
fork()	multiprocessing.Process(target=child_process) and process.start() create the child process (Windows has no os.fork)
exec()	subprocess.run(["cmd", "/c", "dir"]) starts another program from the child process
getpid()	os.getpid() for the process ID, os.getppid() for the parent process ID
exit()	sys.exit(result.returncode) ends the child with an exit status
wait()	process.join() makes the parent wait for the child; process.exitcode gives its exit status
close()	os.close(fd) closes the file descriptor after os.open() and os.write()

Output of Program 2

Run `python 02_system_calls.py` *on your Windows computer and paste the output here. The process IDs and the folder listing shown by* `dir` *are specific to your machine.*